

Propel MVP Build Log

Checkpoint: FL Checkpoint 1 - Working MVP

Core job: Convert one complete proposal form into a grounded client proposal and a downloadable PDF.

Platform: React/Vite frontend, Node server, Groq API, jsPDF.

Log date: 6 August 2026

This log records the actual iteration performed on the submitted codebase. It distinguishes completed work from the one item that must still be produced by the owner: the raw screen capture.

Entry 1 - Narrowed the MVP loop

Goal

Keep the first checkpoint focused on one end-to-end job rather than additional sales or CRM features.

Starting position

The application already had a mandatory form, AI proposal generation, proposal review, and PDF export. It also contained optional editing and section-regeneration controls.

Decision

The MVP success path was fixed as:

1. Complete the mandatory form.
2. Generate the proposal through Groq.
3. Pass automated proposal checks.
4. Confirm human approval without changing the draft.
5. Download and open the PDF.

What was cut from the checkpoint

- Automatic email sending
- Client negotiation
- Payment collection
- Contract signing
- CRM or database integration
- Production-grade authentication
- Automatic pricing decisions
- A second external tool connection

Why

None of these features is required to prove the core form-to-proposal-to-PDF loop. Adding them would make the checkpoint larger and less reliable.

Result

The checkpoint now has one clear, recordable success path that requires no mid-run hand-editing.

Entry 2 - Made the live tool connection visible

Problem observed

The Groq integration existed in the server code, but the user interface did not visibly prove that a real external service was connected. A reviewer watching the run could see proposal generation, but there was no explicit connection state before the run began.

Change made

- Added `checkGroqConnection()` in `proposal-core.js`.
- Added local route `GET /api/health` in `server.js`.
- Added Vercel Function `api/health.js`.
- The health check makes a server-side authenticated request to Groq's models endpoint.
- Added a live status panel above the form:
 - Live tool connected: Groq API
 - configured model name
 - retry button

- clear disconnected or unconfigured state

Security choice

The browser receives only connection status and model information. The API key remains server-side.

Result

The raw run capture can now show the live Groq connection before the proposal is generated.

Entry 3 - Added a pre-recording connection verifier

Problem observed

A missing, expired, or invalid API key could cause the recording to fail after the form had already been completed.

Change made

- Added `scripts/verify-live-connection.mjs`.
- Added command:

```
npm run verify:live
```

- Added combined readiness command:

```
npm run checkpoint:ready
```

This runs the dependency-free core test suite and then performs a real Groq connection check using the private `.env` key.

Result

The owner can verify the live tool immediately before recording instead of discovering an API problem during the capture.

Entry 4 - Expanded the core test suite

Goal

Cover the new live-connection behaviour without making automated tests call the real provider.

Change made

Three Node tests were added:

1. Unconfigured key returns a safe disconnected state.
2. Mocked Groq response verifies the configured model.
3. Provider authentication failure returns a safe disconnected state.

A component test was also added for the visible `Live tool connected: Groq API` status.

Test result

```
14 tests run
14 passed
0 failed
```

The command used was:

```
npm run test:core
```

Result

The core validation, proposal guardrails, controlled regeneration, and connection-check logic pass the dependency-free Node test suite.

Entry 5 - Dependency installation problem in the build environment

Action attempted

```
npm ci
```

What broke

The available package registry returned:

404 Not Found - @playwright/test@~1.54.1

What changed

No dependency version was changed merely to accommodate this restricted build environment. The lockfile was preserved. The dependency-free Node suite was run instead, and JavaScript syntax checks were run on the modified server files and scripts.

Reason

The failure came from the environment's internal package mirror, not from application logic. Replacing or removing Playwright would weaken the intended browser test setup.

Remaining verification

On a normal computer with public npm access, run:

```
npm ci
npm run test:run
npx playwright install chromium
npm run test:e2e
npm run build
```

Entry 6 - Documented the earlier concept change

Earlier concept

The first design concept was a notes-grounded AI study coach.

Deviation

Before the MVP checkpoint, the project was changed to a form-based AI client proposal and PDF generation agent.

Reason

- A working proposal-generator codebase already existed.
- It could be completed with zero additional platform cost.
- It offered a clearer end-to-end tool loop: structured form, live Groq generation, automated checks, human approval, and PDF creation.
- It fit the available build time better than creating a separate study-coach system.

Spec status

The current `CAPSTONE_AGENT_DESIGN.md` and PDF describe the implemented proposal agent. This build log preserves the reason for the deviation rather than hiding it.

Entry 7 - Submission preparation

Added

- `BUILD_LOG.md`
- `BUILD_LOG.pdf`
- `CHECKPOINT_1_SUBMISSION.md`
- Visible Groq health status
- Live connection verification script
- Updated README, testing notes, and change summary

Not included

The raw, unedited screen capture is not included because it must show the owner's real local or deployed run with the owner's private API key.

Current checkpoint state

Requirement	State
Core job works in code from form to PDF	Implemented
Live external tool integrated	Implemented through Groq
Live connection visibly verifiable	Implemented through <code>/api/health</code>
FL-06 deviation documented	Implemented
Real iteration recorded	Implemented in this log
Core automated tests	14 of 14 passed
Raw unedited run capture	Owner must record
Real Groq call verified in this build environment	Not run because no private API key was supplied